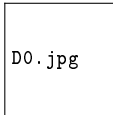


Lecture 24: Supercharging SELECT

Sorting, Filtering, and Finding Patterns

Z2004: Database Management Systems



IIT Madras Zanzibar

Monday, 27 April 2026

Making Queries Work for You!

- ➊ **Pop Quiz:** Let's review Friday's DML concepts!
- ➋ **Removing Clutter:** The `DISTINCT` keyword.
- ➌ **Sorting Data:** Using `ORDER BY`.
- ➍ **Top Results:** Using `TOP`.
- ➎ **SQL Detective:** Pattern searching with `LIKE`.

Mini-Quiz: Test Your DML Knowledge!

Question 1: What happens if you run this query?

```
DELETE FROM Student;
```

Question 2: What is the missing keyword to make this safe?

```
UPDATE Course  
SET Credits = 4  
----- CID = 'DB101';
```

Discuss with your neighbor for 60 seconds!

Mini-Quiz: Answers!

Answer 1: It deletes **EVERY SINGLE ROW** in the Student table! Always use a WHERE clause with DELETE unless you want an empty table.

Answer 2:

```
UPDATE Course
SET Credits = 4
WHERE CID = 'DB101';
```

The WHERE keyword ensures only the Database course gets updated to 4 credits.

1. Removing Clutter: DISTINCT

Sometimes a table has repeating values, and you just want a unique list.

Without DISTINCT:

```
SELECT Dept FROM Student;
```

Dept
CSE
EEE
ME
CSE
CE

(Notice CSE repeats!)

With DISTINCT:

```
SELECT DISTINCT Dept  
FROM Student;
```

Dept
CE
CSE
EEE
ME

DISTINCT vs UNIQUE (don't confuse them)

- ▶ **DISTINCT** is a **query feature**: it removes duplicates **from the result output**.
- ▶ **UNIQUE** is a **schema constraint**: it prevents duplicates **from being stored**.

Feature	DISTINCT	UNIQUE
Where you use it	SELECT DISTINCT ...	CREATE/ALTER TABLE ... CONSTRAINT ... UNIQUE(...)
What it changes	Only the query output	The data rules of the table
NULL behaviour (SQL Server)	Multiple NULLs collapse to one NULL in the output	Allows multiple NULLs (NULL $\neq$ NULL)

DISTINCT example

```
SELECT DISTINCT Dept
FROM Student;
```

UNIQUE example

```
ALTER TABLE Student
ADD CONSTRAINT UQ_Student_Email
UNIQUE (Email);
```

2. Sorting Data: ORDER BY

By default, SQL returns rows in whatever random order it finds them. We can force it to sort our data!

```
-- Sort by GPA (Highest to Lowest)
SELECT Name, GPA
FROM Student
ORDER BY GPA DESC;
```

Name	GPA
Noah Kim	3.92
Asha Ali	3.80
Lina Zhang	3.74
...	...

```
-- Sort Alphabetically (A to Z)
SELECT Name, Dept
FROM Student
ORDER BY Name ASC;
```

Name	Dept
Arjun Singh	CSE
Asha Ali	CSE
Fatima Noor	CE
...	...

3. Finding the Best: TOP

Often, we don't want to see 10,000 rows. We just want the top 3! In SQL Server, we use the TOP keyword right after SELECT.

Goal: Find the 3 Students with the lowest GPAs.

```
SELECT TOP 3 Name , GPA
FROM Student
ORDER BY GPA ASC;
```

Name	GPA
Samuel Mensah	2.95
Hassan Idris	3.05
Maryam Hassan	3.12

4. The SQL Detective: LIKE

What if you remember a student's name started with "A", but you forgot the rest? Use the LIKE operator with the wildcard symbol (%).

```
-- % means "any characters can go here"  
SELECT Name, Dept  
FROM Student  
WHERE Name LIKE 'A%';
```

Name	Dept
Asha Ali	CSE
Arjun Singh	CSE

More Examples:

- ▶ LIKE '%son' → Ends with 'son' (Johnson, Jackson)
- ▶ LIKE '%ali%' → Contains 'ali' anywhere (Alice, Khalid, Malia)

Run these on your existing **Student** table in **SSMS / VS Code**.

- ➊ **DISTINCT**: List all unique departments.
- ➋ **ORDER BY**: Sort students by GPA (high $\rightarrow$ low). Then sort by Name (A $\rightarrow$ Z).
- ➌ **TOP**: Show the bottom 3 GPAs (lowest first).
- ➍ **LIKE**: Find names that (i) start with A, (ii) end with son, (iii) contain ali.

Tip: After each query, also run `SELECT COUNT(*) ...` to confirm how many rows you got.

Self-check: Expected Outcomes

- ▶ **DISTINCT Dept:** you should get **4 rows** (CE, CSE, EEE, ME).
- ▶ **ORDER BY GPA DESC:** highest GPA appears first (e.g., Noah Kim 3.92).
- ▶ **ORDER BY Name ASC:** alphabetical order (e.g., Arjun Singh before Asha Ali).
- ▶ **TOP 3 ... ORDER BY GPA ASC:** returns the **3 lowest** GPAs.
- ▶ **LIKE 'A%':** returns names starting with A (e.g., Asha Ali, Arjun Singh).

Edge Cases & Common Mistakes (SQL Server)

- ▶ **DISTINCT applies to the whole row:** `SELECT DISTINCT Dept, GPA` de-duplicates *pairs*, not just `Dept`.
- ▶ **ORDER BY with ties:** add a second key (e.g., `ORDER BY GPA DESC, Name ASC`).
- ▶ **TOP without ORDER BY is meaningless:** always sort first.
- ▶ **TOP WITH TIES:** includes extra rows if there's a tie at the cutoff.
- ▶ **LIKE wildcards:** `%` = any length, `_` = exactly one character.
- ▶ **Case sensitivity:** depends on your database collation (many SQL Server setups are case-insensitive).
- ▶ **Escaping:** to match a literal `%` or `_`, use `ESCAPE` (e.g., `LIKE '%_%' ESCAPE '\'`).

Key takeaways (L24: Supercharging SELECT)

- ▶ **DISTINCT**: Removes duplicate rows from your results.
- ▶ **ORDER BY**: Sorts your data. Use **ASC** for going up (A-Z, 1-10) and **DESC** for going down (Z-A, 10-1).
- ▶ **TOP n**: Chops off the results to only show a specific number of rows (great for leaderboards).
- ▶ **LIKE**: Uses the % wildcard to search for partial text matches.

Tomorrow: Using SQL as a Calculator!